Computation-aware Streaming Speech Translation on a Raspberry Pi 5

Fredrik Boglind

Project Report in
Tiny Machine Learning
Autumn term 2026
Instructor: Jing Zhan
Department of Computer
and Systems Sciences

Stockholm
University

Abstract

Streaming speech translation must balance translation quality against the delay before translated text becomes available. This project investigates that trade-off on a Raspberry Pi 5, focusing on the effect of computation time in a resource-constrained environment. An English-to-German cascade combining Whisper automatic speech recognition with Marian machine translation was evaluated using two ASR model sizes and two fixed audio chunk durations (4 and 8 seconds). The same configurations were tested on a laptop CPU for comparison. Experiments covered six recordings from the MCIF development set, comprising approximately 33.6 minutes of speech. Evaluation included translation quality, word error rate, computation-aware and computation-unaware latency, processing real-time factor (RTF), and resource use. All Raspberry Pi configurations achieved an aggregate processing RTF below 1, although temporary processing queues occurred. Longer chunks generally improved translation quality at the cost of increased latency. Accounting for computation increased measured latency substantially, particularly on the Pi. Supplementary experiments examined a smaller ASR model and a LocalAgreement prototype. The results demonstrate that the tested cascade can maintain sufficient average processing throughput on the Raspberry Pi 5. However, the measured latency, temporary processing stalls and recognition errors limit what can be concluded about its suitability for interactive live translation.

Abbreviations

Abbreviation	Description
API	Application Programming Interface
ASR	Automatic Speech Recognition
CA	Computation-Aware (latency)
chrF2	Character n-gram F-score ($\beta = 2$)
COMET	Crosslingual Optimized Metric for Evaluation of Translation
CPU	Central Processing Unit
CU	Computation-Unaware (latency)
GPU	Graphics Processing Unit
INT8	8-bit integer representation
IWSLT	International Conference on Spoken Language Translation
LLM	Large Language Model
LongYAAL	Long-form extension of Yet Another Average Lagging
MCIF	Multimodal Crosslingual Instruction-Following
MT	Machine Translation
PCM	Pulse-Code Modulation
RSS	Resident Set Size
RTF	Real-Time Factor
SimulST	Simultaneous Speech Translation
WER	Word Error Rate
YAAL	Yet Another Average Lagging

List of Figures

4.1 Translation quality and latency for the main configurations on both devices. Hollow markers represent computation-unaware LongYAAL and filled markers represent computation-aware LongYAAL. Arrows connect measurements for the same configuration. The upper panels show chrF2 and the lower panels show COMET. Values are aggregated over six recordings, with timing averaged across two repetitions. 13
4.2 Audio-weighted processing real-time factor (RTF) for ASR and MT on the Raspberry Pi and laptop. The labels show total processing RTF, including small additional processing overhead. The dashed line indicates RTF = 1. Model loading, warm-up and deliberate waiting for audio are excluded. . . . 15
4.3 Two transient queue events observed on the Raspberry Pi, with the corresponding laptop recording and configuration shown for comparison. The figure displays the first repetition. Queue waiting times are associated with chunk audio-end timestamps; generated outputs differ between devices. . 16

List of Tables

3.1 Decoding and model parameters. 7
3.2 Experimental factors and their values. 9

4.1 Main experiment over six talks (33.63 minutes, 266 reference segments). Quality is corpus chrF2, segment-mean COMET and pooled WER. Timing is the mean of two complete passes; RTF is audio-weighted processing time, excluding deliberate playback waits. CU and CA are LongYAAL in seconds, with clock-specific eligible segments. The repeated outputs were identical within each device/configuration. 12
4.2 First is the mean first-output time from recording onset. Tail and queue values are maxima across all 12 runs per configuration, including shorter final chunks. RSS is the maximum sampled replay process RSS; resources were sampled every five seconds plus start/end, so brief memory peaks may be missed. Loading and warm-up are outside replay timing. . . 15
4.3 Supplementary Whistle cascade results on the same six recordings as the main experiment. Quality scores are corpus-level or pooled measures, while CA LongYAAL and processing RTF are averaged over two repetitions. 18

Contents

Abstract ii

Abbreviations iii

List of Figures iv

List of Tables v

Contents vi

1 Introduction 1

2 Extended Background 3
- 2.1 Streaming Speech Translation . . . 3
- 2.2 Evaluating Translation Quality and Latency . . . 4
- 2.3 Speech Translation on Edge Devices . . . 4

3 Method 6
- 3.1 System Architecture . . . 6
- 3.2 Hardware and Software . . . 6
- 3.3 Models and Inference Configuration . . . 7
- 3.4 Evaluation Data . . . 8
- 3.5 Experimental Procedure . . . 8
- 3.6 Evaluation Metrics . . . 9
- 3.7 Supplementary Evaluation . . . 10

4 Results 12
- 4.1 Translation Quality and Latency . . . 13
- 4.2 Processing Throughput and Queue Behaviour . . . 14
- 4.3 Recognition Errors and Output Consistency . . . 16
- 4.4 Exploratory Translation Quality Control . . . 17
- 4.5 Supplementary Whistle Comparison . . . 18

5 Discussion **20**
5.1 Feasibility and the Quality–Latency Trade-off 20
5.2 Recognition Errors and Translation Quality 22
5.3 Exploratory Extensions 23
5.3.1 Whistle as an Alternative ASR Model 23
5.3.2 LocalAgreement Prototype 23
5.4 Limitations and Future Work 24

6 Conclusion **26**

Bibliography **29**

1

Introduction

Simultaneous speech-to-text translation (SimulST) aims to produce written translations while the source speech is still being delivered. Unlike offline speech translation, a streaming system must decide when enough information has been received to generate and release a translation. Waiting for more speech generally provides additional context, but also increases the delay before translated text becomes available. Balancing translation quality and latency is therefore a central challenge in SimulST [6].

Latency depends not only on how much speech a system collects before producing output, but also on the time required to process it. This distinction is reflected in *computation-unaware* latency, which assumes that model inference is instantaneous, and *computation-aware* latency, which accounts for actual processing time. The IWSLT 2026 simultaneous speech translation track reports both measurements, although its latency regimes are determined using computation-unaware LongYAAL [3]. On resource-constrained hardware, however, computation time may contribute substantially to the delay experienced by a user.

Running speech translation locally on an edge device may be useful when network connectivity is limited or speech data should remain on the device. However, such devices typically have considerably less computational capacity than the hardware used to evaluate larger translation systems. This raises questions about whether local streaming translation can keep pace with incoming speech, and how the available computational resources affect the trade-off between quality and latency.

This project investigates these questions by implementing an English-to-German streaming speech translation system on a Raspberry Pi 5. The system uses a cascade of pretrained Automatic Speech Recognition (ASR) and machine translation (MT) models, processing incoming audio in successive fixed-length chunks. Four configurations, combining two ASR model sizes with two chunk lengths, are evaluated on the Raspberry Pi and compared with the same configurations running on a laptop CPU.

The study addresses two research questions:

1. How does including computation time change the observed quality–latency trade-off across ASR model sizes and audio chunk lengths on the Raspberry Pi 5?

2. Which tested configurations can keep up with incoming speech, and how does their performance compare with the same configurations on a laptop CPU?

The evaluation uses approximately 34 minutes of recorded scientific presentations, assessing translation quality, recognition errors, latency, processing speed, and resource use. Selected output errors are also examined to help interpret the quantitative findings. The aim is to evaluate the practical feasibility and limitations of this particular implementation, rather than to develop a new translation model or establish performance across edge devices generally.

2

Extended Background

2.1 Streaming Speech Translation

Speech translation systems can broadly be divided into *direct* and *cascaded* approaches. Direct systems translate speech into target-language text using a single model, whereas cascaded systems first perform automatic speech recognition (ASR) and then translate the resulting text using a machine translation (MT) model. Although recognition errors may propagate into the translation stage, modern cascaded systems can achieve translation quality comparable to direct systems [1].

In simultaneous speech translation, the system must produce output before the entire source recording is available. This requires a *streaming policy* that determines how much incoming audio to process and when to release translations. Many pretrained speech models were originally developed for offline processing and require additional mechanisms to operate incrementally. For example, Macháček, Dabre, and Bojar [5] adapted Whisper for streaming transcription using a so-called LocalAgreement policy, which identifies stable output across successive predictions.

A simpler approach processes successive fixed-length audio chunks independently, producing a translation after each chunk has been processed. Shorter chunks allow processing to begin earlier but provide less linguistic context and may divide words or sentences. Longer chunks provide more context at the cost of additional waiting time. This fixed-chunk approach forms the baseline investigated in the present project.

2.2 Evaluating Translation Quality and Latency

Evaluating streaming speech translation requires considering both translation quality and the delay before translated text becomes available. These objectives can conflict: waiting for additional speech may improve the translation, while releasing output earlier reduces latency.

An important distinction is between *computation-unaware* (CU) and *computation-aware* (CA) latency. CU latency measures delay under the assumption that processing takes no time, primarily reflecting the system's decisions about when to produce output. CA latency instead uses actual output times, incorporating the time required for inference and any accumulated processing delays. As Papi et al. [6] discuss, computation-aware measurements are particularly relevant when assessing the practical performance of streaming systems.

Evaluating continuous speech translation introduces additional challenges because the system's output segmentation may differ from that of the reference translations. Polák et al. [9] introduced LongYAAL, an extension of Yet Another Average Lagging (YAAL) designed for evaluating latency in long-form speech translation. Their OmniSTEval toolkit provides alignment and resegmentation procedures for evaluating continuous translation output [8]. LongYAAL is also used in the IWSLT 2026 simultaneous speech translation track [3].

Translation quality can be assessed separately using reference-based metrics such as chrF and COMET. In a cascaded system, the intermediate ASR output can additionally be evaluated against a source-language transcript. This makes it possible to investigate whether poor translation quality is associated with speech recognition errors or errors introduced during translation.

2.3 Speech Translation on Edge Devices

Deploying speech translation on edge hardware introduces computational constraints that may be less significant on powerful GPU-based systems. Devices such as the Raspberry Pi 5 have limited CPU and memory resources, making inference efficiency an important consideration.

One approach to reducing computational requirements is quantisation, in which model weights are represented using lower numerical precision. Inference frameworks such as CTranslate2 support quantised execution on

CPUs, including mixed computation types such as `int8_float32`. The actual performance depends on the model, hardware architecture and inference implementation.

A commonly used measure of computational throughput is the *Real-Time Factor* (RTF), defined as processing time divided by audio duration. An RTF below 1 indicates that processing takes less time than the corresponding audio duration. However, average throughput does not fully describe streaming performance. Individual chunks may require unusually long processing times, temporarily delaying subsequent chunks even when the system processes a complete recording faster than real time.

Consequently, evaluating streaming translation on resource-constrained hardware requires examining translation quality, computation-aware latency and processing behaviour together. This motivates the measurements used in the present study.

3

Method

3.1 System Architecture

The system implements a cascaded English-to-German speech translation pipeline. Incoming English audio is processed by an ASR model, and the resulting English transcription is translated into German using a separate MT model. Both stages run locally on the CPU.

To simulate streaming input, prerecorded audio is replayed at its original speed. The system processes successive, non-overlapping audio chunks of fixed duration. A chunk becomes available only after all its audio samples have arrived. ASR and MT are executed sequentially, and the complete German translation of each chunk is emitted when MT finishes.

The system uses a single processing worker. If one chunk is still being processed when the next becomes available, the subsequent chunk waits in a queue. No chunks are discarded. There is no voice activity detection, overlapping context, conditioning on previous transcriptions, or revision of previously emitted text. This deliberately simple streaming policy allows the effects of chunk length, ASR model size, and computation time to be investigated.

3.2 Hardware and Software

Experiments were conducted on two CPU-based devices:

- **Raspberry Pi 5:** Model B with a quad-core ARM Cortex-A76 processor, 8 GB RAM, active cooling, and Debian 13.

- **Laptop:** Lenovo ThinkPad T480s with an Intel Core i7-8650U processor (four physical cores, eight logical threads), 24 GB RAM, and Ubuntu 24.04.

The Raspberry Pi and laptop used Python 3.13.5 and 3.12.3, respectively. Both used faster-whisper 1.2.1 and CTranslate2 4.8.2 for inference. Four inference threads were configured for both devices.

The Pi was operated with its cooling fan enabled, while the laptop was connected to external power with its performance profile set to balanced. The experiments were run without intentionally competing computational workloads. Hardware information, software versions, and resource measurements were recorded alongside the experiment outputs.

3.3 Models and Inference Configuration

Two pretrained English Whisper models were compared for ASR: `tiny.en` and `base.en` [11]. Both were executed using faster-whisper [13], which uses the CTranslate2 inference engine [4].

Machine translation was performed using the pretrained Helsinki-NLP `opus-mt-en-de` model [14]. The original Marian model was converted to CTranslate2 with INT8 quantisation and held fixed throughout the main experiment.

The effective CTranslate2 computation type was recorded as `int8_float32` for both ASR and MT on both devices. Quantisation was therefore part of the inference setup, not an experimental variable.

The main decoding settings are shown in Table 3.1

Table 3.1: Decoding and model parameters.

Parameter	Value
ASR models	Whisper `tiny.en`, `base.en`
MT model	Helsinki-NLP `opus-mt-en-de`
Audio chunk duration	4 or 8 seconds
Inference threads	4
ASR beam size	1
ASR temperature	0
MT beam size	4
MT maximum output	256 tokens
Effective computation type	`int8_float32`
Previous-text conditioning	Disabled

All models were loaded and warmed up before each recording was replayed. Model loading and warm-up times were logged separately and excluded from the primary processing-time measurements. No models were trained or fine-tuned as part of the project.

3.4 Evaluation Data

The evaluation data were drawn from MCIF (Multimodal Crosslingual Instruction Following), a multilingual dataset based on scientific presentations [7]. The recordings contain English speech accompanied by English transcripts and German reference translations.

An initial pilot used one recording to test the complete inference and evaluation pipeline and establish the configurations for the main experiment.

For the main evaluation, six different recordings were selected using a deterministic procedure based on recording identifiers and durations, without inspecting system predictions or translation scores. The pilot recording was excluded.

The resulting evaluation set contained approximately **33.63 minutes of audio and 266 reference segments**. The same prepared recordings were used on both devices. Audio was supplied as mono, 16 kHz, 16-bit PCM.

The reference transcripts, translations, and segment boundaries were used for evaluation only. They were not provided to the inference system during streaming.

3.5 Experimental Procedure

The main experiment followed a factorial design with two ASR model sizes and two chunk durations. Each configuration was evaluated on all six recordings, twice on each device as shown in Table 3.2.

Each run processed one complete recording in a separate inference process. A monotonic clock controlled audio availability and recorded when each chunk became ready, when processing started, and when its German translation was emitted.

The second repetition reversed the configuration and recording order to reduce systematic effects associated with execution order. The two repetitions used identical audio and model settings. They provided repeated timing measurements rather than additional independent evaluation data.

Table 3.2: Experimental factors and their values.

Factor	Values
Device	Raspberry Pi 5, laptop
ASR model	`tiny.en`, `base.en`
Chunk duration	4 s, 8 s
Recordings	6
Repetitions	2
Total runs	**96**

For every chunk, the system recorded ASR and MT processing durations, queue waiting time, and output timestamps. Resource measurements were collected during replay, including process memory consumption, temperature, and available throttling indicators. Resource sampling was performed approximately every five seconds, with additional measurements at the beginning and end of replay.

3.6 Evaluation Metrics

The evaluation considered translation quality, recognition accuracy, latency, processing throughput, and resource use.

Translation quality. German translations were evaluated against the official reference translations using chrF2 [10] and COMET [12]. chrF2 measures character-level overlap with the reference, while COMET uses a learned model to evaluate translations in relation to source and reference text. The `wmt22-comet-da` model was used, with the official English transcript as its source input.

ASR accuracy. English recognition output was evaluated using word error rate (WER), calculated from substitutions, deletions, and insertions relative to the English reference transcript. Text was lowercased, punctuation was removed, and whitespace was normalised before scoring.

Latency. Translation latency was measured using computation-unaware (CU) and computation-aware (CA) LongYAAL, implemented through OmniSTEval 0.1.10 [8]. CU timestamps represent the time at which the corresponding audio chunk became available, while CA timestamps represent

the actual completion of translation, including processing and queue delays.

The continuous German output was aligned and resegmented against the reference segments before calculating quality and latency metrics. Latency was evaluated using the actual recording endpoint. Because the metric may exclude different output portions under the two clocks, the difference between CA and CU LongYAAL should not be interpreted as a direct measurement of computation time.

Processing throughput. Real-time factor (RTF) was calculated as total processing time divided by audio duration:

$$\mathrm{RTF} = \frac{\text{total processing time}}{\text{audio duration}} \tag{3.1}$$

Processing time excludes deliberate waiting for incoming audio, model loading, and warm-up. ASR and MT stage durations were recorded separately.

Additional timing diagnostics included queue waiting times, time to first translated output, and the delay between the end of the recording and completion of the final translation.

Resource use. Process memory consumption and system temperature were sampled during inference. Raspberry Pi throttling flags were recorded before and after replay. Energy consumption was not measured.

For aggregate results, chrF2 was calculated at corpus level, COMET was averaged over reference segments, WER was pooled over reference words, and RTF was weighted by audio duration. LongYAAL was averaged over eligible reference segments. The two complete repetitions were then summarised using their mean values and observed ranges.

During evaluation, an issue involving empty resegmented predictions required a correction to the evaluation adapter. Missing timestamp arrays were replaced with empty arrays only for predictions containing no words. The original inference outputs were preserved, and empty translations remained included in quality evaluation.

3.7 Supplementary Evaluation

Two additional analyses were conducted separately from the main experiment.

First, an exploratory translation quality-control procedure examined selected passages using automated language-model judgments and limited manual inspection. The procedure compared recognised English text with German translations to identify possible omissions, meaning changes, terminology errors, and unsupported additions. The initial screening used Jev 1.13 (build 20260917), TypeSafe's structured decision model [15]. A subset of cases was subsequently reviewed using Claude Sonnet 4.5. The automated screening questions and validation rules are provided in the accompanying quality-control notebook. Because the sample was selected partly on the basis of model-generated scores, these judgments were treated as diagnostic observations rather than estimates of overall error frequency.

Second, Whistle, an open-weight ASR model released by Cactus Compute in October 2026 [2], was evaluated as a supplementary alternative to Whisper. Whistle replaced the ASR component while retaining the fixed 4- and 8-second chunking policy and the existing Marian translation model. Both devices processed the same six recordings with two repetitions, producing 48 additional runs. Evaluation used the same automatic quality, latency, and throughput measures as the main experiment. Since Whistle also differs in inference backend, quantisation, and decoding controls, this comparison evaluates complete ASR implementations rather than isolating model architecture.

The LocalAgreement prototype was evaluated separately on a short recording and was not included in the main or Whistle experimental grids.

4

Results

The main experiment comprised 96 completed runs across two devices, two ASR models, two chunk durations and two repetitions of six recordings. Table 4.1 summarises the translation quality, recognition accuracy, latency and processing throughput. Quality scores were identical across repetitions within each device and configuration; the reported timing measurements are averages over the two repetitions.

Device	ASR / chunk	chrF2	COMET	WER (%)	CU (s)	CA (s)	RTF
pi	tiny.en / 4s	46.68	0.559	36.02	2.15	3.53	0.322
pi	tiny.en / 8s	50.74	0.626	25.61	4.20	6.04	0.228
pi	base.en / 4s	50.26	0.602	26.36	2.12	4.20	0.516
pi	base.en / 8s	52.28	0.659	22.92	4.28	7.12	0.350
laptop	tiny.en / 4s	45.53	0.556	40.91	2.19	2.59	0.097
laptop	tiny.en / 8s	50.52	0.629	26.18	4.20	4.67	0.059
laptop	base.en / 4s	49.92	0.603	26.95	2.13	2.82	0.173
laptop	base.en / 8s	51.31	0.650	29.39	4.22	5.02	0.100

Table 4.1: Main experiment over six talks (33.63 minutes, 266 reference segments). Quality is corpus chrF2, segment-mean COMET and pooled WER. Timing is the mean of two complete passes; RTF is audio-weighted processing time, excluding deliberate playback waits. CU and CA are LongYAAL in seconds, with clock-specific eligible segments. The repeated outputs were identical within each device/configuration.

4.1 Translation Quality and Latency

Figure 4.1 shows the relationship between translation quality and latency under computation-unaware (CU) and computation-aware (CA) evaluation. Accounting for computation increased measured latency for every configuration, with larger differences on the Raspberry Pi than on the laptop.

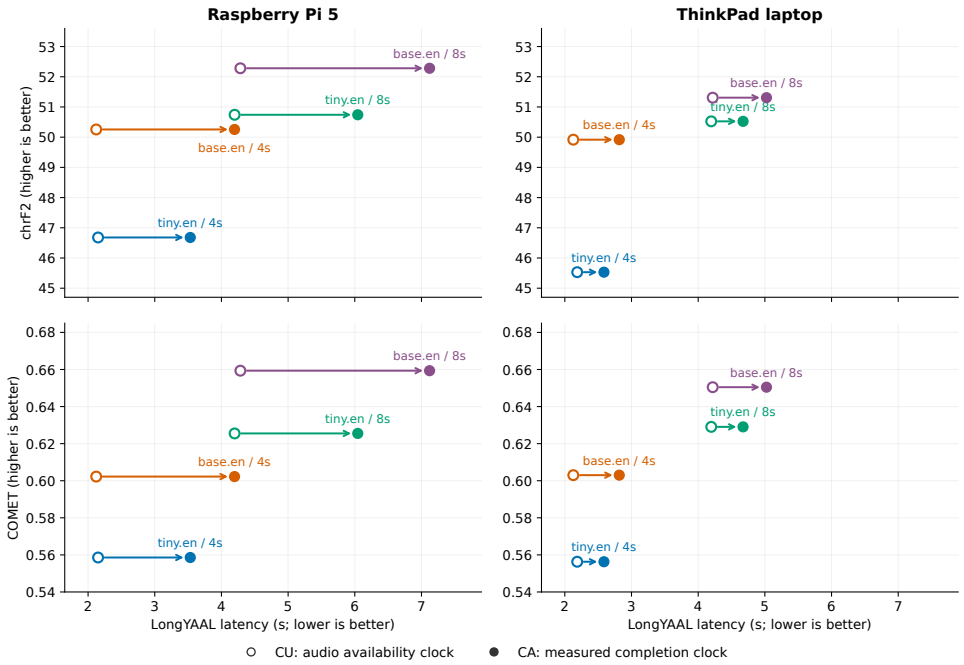


Figure 4.1: Translation quality and latency for the main configurations on both devices. Hollow markers represent computation-unaware LongYAAL and filled markers represent computation-aware LongYAAL. Arrows connect measurements for the same configuration. The upper panels show chrF2 and the lower panels show COMET. Values are aggregated over six recordings, with timing averaged across two repetitions.

On the Pi, computation-unaware LongYAAL ranged from 2.12 to 4.28 seconds, compared with 3.53 to 7.12 seconds when computation was included. The differences between CA and CU ranged from 1.38 seconds for `tiny.en` with 4-second chunks to 2.84 seconds for `base.en` with 8-second chunks. These are differences between latency metrics rather than direct measurements of inference duration.

Accounting for computation also changed the relative latency of the two ASR models. With 4-second chunks, the models had nearly identical CU latency on the Pi: 2.15 seconds for `tiny.en` and 2.12 seconds for `base.en`. Under CA evaluation, however, `tiny.en` had lower latency (3.53

versus 4.20 seconds). Thus, the computational cost of the larger model was largely absent from the CU comparison.

Increasing chunk duration from 4 to 8 seconds improved both translation-quality metrics for both ASR models on both devices. On the Pi, `tiny.en` improved from 46.68 to 50.74 chrF2 and from 0.559 to 0.626 COMET. For `base.en`, chrF2 increased from 50.26 to 52.28 and COMET from 0.602 to 0.659. Similar improvements were observed on the laptop.

These quality improvements were accompanied by increased latency. On the Pi, CA LongYAAL increased from 3.53 to 6.04 seconds for `tiny.en` and from 4.20 to 7.12 seconds for `base.en`. The highest translation quality was obtained with `base.en` and 8-second chunks, while `tiny.en` with 4-second chunks had the lowest measured CA latency.

Translation-quality differences between devices were relatively small. Across the four configurations, Pi chrF2 scores were 0.22–1.15 points higher than the corresponding laptop scores, while COMET differences ranged from approximately -0.004 to +0.009. The generated translations were not identical across devices, however, as examined in Section 4.3.

4.2 Processing Throughput and Queue Behaviour

All tested configurations processed the recordings faster than real time overall. Mean processing RTF ranged from 0.228 to 0.516 on the Raspberry Pi and from 0.059 to 0.173 on the laptop. For corresponding configurations, processing on the Pi took approximately 2.99–3.88 times as long as on the laptop.

Figure 4.2 separates processing time into ASR and MT components. On both devices, 8-second chunks required less processing time per second of audio than 4-second chunks for the same ASR model. Nevertheless, the 8-second configurations had higher computation-aware latency, reflecting the additional time spent collecting audio before processing could begin.

Average throughput did not imply that every chunk completed before the next became available. Table 4.2 reports first-output times, maximum queue waits, final processing tails and sampled memory consumption.

The largest observed queue wait on the Pi was 8.85 seconds with `tiny.en` and 4-second chunks. Inspection of the recorded chunk logs identified a long repetition of the word “route”. The affected chunk required 12.77 seconds of processing, delaying subsequent chunks.

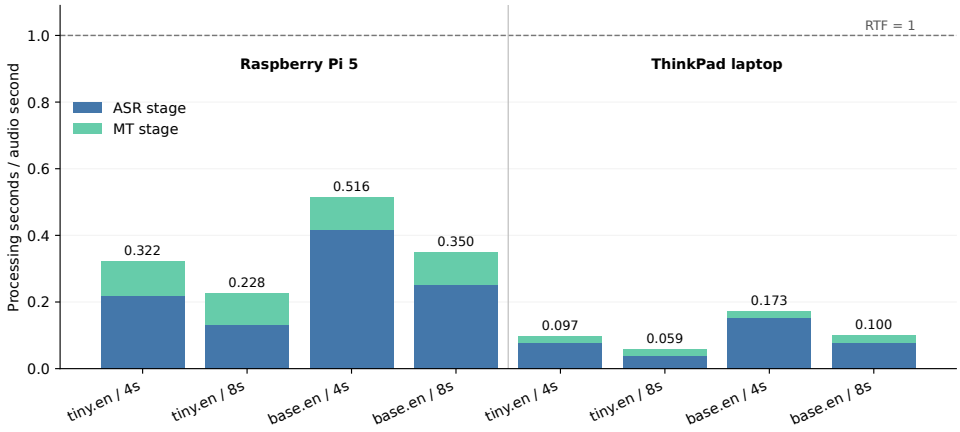


Figure 4.2: Audio-weighted processing real-time factor (RTF) for ASR and MT on the Raspberry Pi and laptop. The labels show total processing RTF, including small additional processing overhead. The dashed line indicates RTF = 1. Model loading, warm-up and deliberate waiting for audio are excluded.

Device	ASR / chunk	First (s)	Tail max (s)	Queue max (s)	RSS (MiB)
pi	tiny.en / 4s	5.37	1.98	8.85	461.9
pi	tiny.en / 8s	9.92	2.63	1.97	419.5
pi	base.en / 4s	6.16	3.40	1.95	506.1
pi	base.en / 8s	10.89	4.30	6.31	532.2
laptop	tiny.en / 4s	4.40	0.51	0.26	446.7
laptop	tiny.en / 8s	8.49	0.64	0.39	417.7
laptop	base.en / 4s	4.69	1.09	0.61	480.5
laptop	base.en / 8s	8.80	1.22	0.75	526.6

Table 4.2: First is the mean first-output time from recording onset. Tail and queue values are maxima across all 12 runs per configuration, including shorter final chunks. RSS is the maximum sampled replay process RSS; resources were sampled every five seconds plus start/end, so brief memory peaks may be missed. Loading and warm-up are outside replay timing.

A second event occurred with `base.en` and 8-second chunks, where repeated quantities in the ASR output were associated with a 14.31-second processing step and a subsequent queue wait of 6.31 seconds.

In both cases, the queues subsequently decreased rather than continuing to grow throughout the recording. Figure 4.3 illustrates these events alongside the corresponding laptop runs.

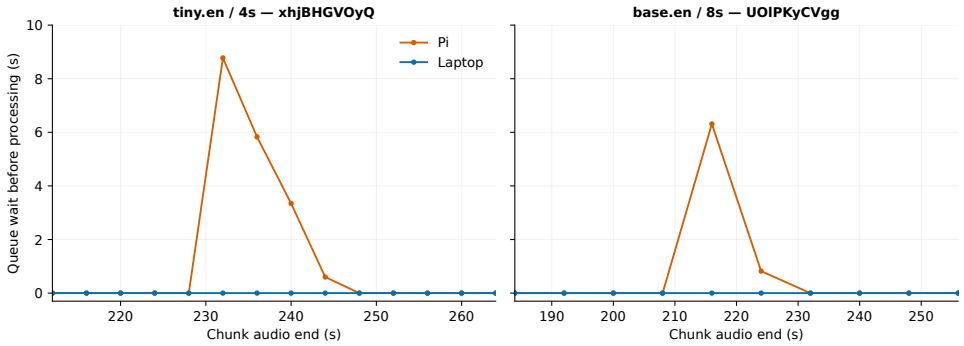


Figure 4.3: Two transient queue events observed on the Raspberry Pi, with the corresponding laptop recording and configuration shown for comparison. The figure displays the first repetition. Queue waiting times are associated with chunk audio-end timestamps; generated outputs differ between devices.

Mean first-output times on the Pi ranged from 5.37 to 10.89 seconds, compared with 4.40–8.80 seconds on the laptop. The longer chunk configurations generally produced their first translation later.

Maximum sampled replay process memory ranged from approximately 419 to 532 MiB on the Pi and 418 to 527 MiB on the laptop. The highest sampled Pi temperature was 68.85 °C. All Pi runs recorded a throttling status of `0x0` before and after replay. No throttling events were identified in the recorded measurements.

4.3 Recognition Errors and Output Consistency

English ASR performance was evaluated separately from German translation quality using word error rate (WER). The results are included in Table 4.1.

On the Pi, increasing chunk duration from 4 to 8 seconds reduced WER for both ASR models. For `tiny.en`, WER decreased from 36.02% to 25.61%, while `base.en` improved from 26.36% to 22.92%. The latter was the lowest WER among the Pi configurations.

The laptop showed the same pattern for `tiny.en`, whose WER decreased from 40.91% to 26.18%. However, laptop `base.en` performed worse with 8-second chunks, increasing from 26.95% to 29.39%.

Further inspection showed that this deterioration was mainly associated with increased insertions. The number of inserted words rose from 318 to 565, while substitutions decreased from 767 to 626. Two recordings accounted for most of the additional insertions, and their outputs contained extensive repetition. The other four recordings showed improved WER with 8-second chunks.

Despite this increase in English recognition errors, laptop `base.en` achieved higher German chrF2 and COMET scores with 8-second chunks. Recognition accuracy and final translation quality therefore did not always change in the same direction.

The two repetitions of each device/configuration combination produced identical English and German text outputs and identical computation-unaware emission schedules. By contrast, all 24 corresponding recording/configuration comparisons between the Pi and laptop produced different ASR and translation outputs. These differences were retained in the evaluation rather than forcing identical predictions across devices.

4.4 Exploratory Translation Quality Control

An exploratory quality-control analysis was conducted to investigate translation errors that were not readily explained by the aggregate quality metrics.

The initial Jev screening examined 96 outputs from a fixed sample of 12 source passages across the eight main device/configuration combinations. Six additional diagnostic outputs were also evaluated.

At a score threshold of 0.5, Jev flagged at least one error category in 81 of the 96 fixed-sample outputs. No meaningful omission scores reached this threshold. The number of flagged outputs was sensitive to the threshold, falling to 52 at 0.7 and zero at 0.9. These scores were not calibrated as probabilities of actual translation errors.

A subsequent review selected 36 outputs for more detailed assessment using a second language model. Of these, 33 produced validated responses and three failed validation or authentication checks.

The two evaluators sometimes disagreed. In the 33 successfully reviewed cases, the detailed reviewer marked three meaningful omissions that had

not been flagged by Jev. There were also disagreements concerning meaning changes and terminology or quantity errors.

Limited manual inspection identified examples of apparently missing content, unclear English recognition and extensive repetition. Some cases remained uncertain, particularly where the English ASR output was itself difficult to interpret.

The analysis provided examples of possible error types and highlighted limitations of the automated judgments. Because the cases were selected partly using model scores and were not independently annotated by expert translators, the findings do not establish population error rates or screening accuracy.

4.5 Supplementary Whistle Comparison

A supplementary experiment replaced Whisper ASR with Whistle while retaining the same six recordings, 4- and 8-second chunk durations, and Marian translation model. The experiment comprised 48 runs across the two devices and two repetitions.

Table 4.3 summarises the results. As in the main experiment, the quality scores were identical across repetitions within each configuration.

Device	Chunk	chrF2	COMET	WER (%)	CA (s)	RTF
Pi	4 s	46.87	0.558	33.42	3.91	0.419
Pi	8 s	50.13	0.607	26.85	7.44	0.401
Laptop	4 s	46.90	0.558	33.48	2.40	0.056
Laptop	8 s	50.04	0.606	26.76	5.41	0.153

Table 4.3: Supplementary Whistle cascade results on the same six recordings as the main experiment. Quality scores are corpus-level or pooled measures, while CA LongYAAL and processing RTF are averaged over two repetitions.

Whistle achieved translation-quality scores broadly comparable to those of the smaller Whisper model. On the Pi, Whistle obtained 46.87 chrF2 with 4-second chunks and 50.13 with 8-second chunks, compared with 46.68 and 50.74 for `tiny.en`.

However, the Whistle cascade did not reduce total Pi processing time. Its RTF was approximately 0.40–0.42, compared with 0.228–0.322 for the corresponding `tiny.en` configurations. Most of the measured Whistle processing time on the Pi occurred in the translation stage rather than ASR.

Sampled process memory use was lower for Whistle, at approximately 259–267 MiB on the Pi, compared with 419–532 MiB for the main Whisper configurations.

The laptop results varied between chunk durations: RTF increased from 0.056 for 4-second chunks to 0.153 for 8-second chunks. One first-pass laptop job was recorded as starting without external power. This deviation was retained in the supplementary results; its contribution to the observed timing variation was not established.

No additional manual or LLM-based quality control was performed for Whistle.

5

Discussion

5.1 Feasibility and the Quality–Latency Trade-off

The results indicate that the tested cascaded speech translation system can process recorded speech at a sufficient average rate to keep up with real-time input on a Raspberry Pi 5. All four main configurations achieved an aggregate processing real-time factor (RTF) below 1, ranging from 0.228 to 0.516. The laptop was considerably faster, with RTF values between 0.059 and 0.173, but the Pi completed all recordings without sustained queue growth or observed thermal throttling.

These results establish that the Pi has sufficient average processing capacity for the tested cascade and recordings. Whether this translates into a practically useful live translation system remains an open question, particularly given the measured latency and observed recognition errors.

However, average throughput alone does not establish consistently low latency. The queue-recovery examples in Figure 4.3 illustrate how individual chunks can take substantially longer to process than the interval between incoming chunks. Although the system recovered in both cases, users would still experience temporary delays. Consequently, an RTF below 1 should be interpreted as evidence of sufficient aggregate processing capacity, rather than a guarantee that every translation arrives on time.

The largest queue event illustrates how an isolated recognition failure can affect both translation quality and responsiveness. In recording xhjB-HGVOyQ, the Pi's tiny.en configuration produced 224 repetitions of the word "route" from a four-second audio chunk. The translation stage subsequently

generated repetitive output up to its 256-token limit. Processing this chunk took 12.77 seconds, causing the next chunk to wait 8.78 seconds before processing could begin. The maximum queue wait across the two repetitions was 8.85 seconds. The backlog subsequently cleared.

This demonstrates why a processing RTF below one is not sufficient to guarantee consistently low latency. Although the system had adequate average throughput, unusually long model outputs temporarily blocked subsequent chunks. Such events motivate investigating repetition detection, output-length safeguards and the distribution of chunk-level delays in addition to aggregate latency.

The distinction between computation-unaware and computation-aware latency was particularly important on the Pi. Computation-unaware LongYAAL (CU) ranged from 2.12 to 4.28 seconds, whereas computation-aware LongYAAL (CA) ranged from 3.53 to 7.12 seconds. The difference was smaller on the laptop. This demonstrates why latency measurements that consider only how much audio has been consumed can present an overly favourable picture of performance on resource-constrained hardware, as also discussed by [6]. The difference between CU and CA should not, however, be interpreted as a direct measurement of inference time, since the metrics also differ in their timing and segment-eligibility conditions.

The experiment also revealed a clear trade-off between chunk duration, translation quality and latency. Increasing the chunk duration from 4 to 8 seconds improved both chrF2 and COMET for every main configuration on both devices. Longer chunks provide the recognition and translation models with more linguistic context, but also require the system to wait longer before producing output. The resulting increase in latency was substantial even when the longer chunks reduced overall processing demand.

Increasing ASR model size produced a different trade-off. On the Pi, `base.en` generally improved recognition and translation quality relative to `tiny.en`, but required more processing time. The highest translation quality was achieved with `base.en` and 8-second chunks, which also had the highest computation-aware latency of the main Pi configurations. No single configuration was therefore best in all respects.

The appropriate configuration depends on the intended application. A system prioritising responsiveness may prefer shorter chunks and a smaller ASR model, while an application that tolerates several seconds of delay may benefit from the additional context and higher translation quality of the larger configuration. The present experiment does not establish a universally optimal operating point, but it makes these practical trade-offs visible.

5.2 Recognition Errors and Translation Quality

The relationship between recognition accuracy and translation quality was not entirely straightforward. Although longer chunks generally improved both, the laptop's `base.en` configuration had a higher WER with 8-second chunks than with 4-second chunks, while its German chrF2 and COMET scores improved.

The increase in WER was primarily associated with insertions, including repeated speech-recognition output in particular recordings. This illustrates a limitation of interpreting an end-to-end speech translation system through a single intermediate metric. Recognition accuracy is important, but the number and type of recognition errors do not map directly onto the quality of the resulting translation. Some errors may have little effect on meaning, while others can substantially alter the translated content.

The exploratory quality-control analysis provided additional information about possible failure modes. Automated screening identified passages worth examining for omissions, changes in meaning, entity or quantity errors, and unsupported additions. However, the number of flagged passages was highly sensitive to the chosen screening threshold, and the two automated evaluators did not always agree.

In particular, the second reviewer identified several possible meaningful omissions that the initial screening had not flagged. Manual inspection also highlighted cases in which the apparent translation problem originated in the English recognition output. Repetition or missing content in the ASR output could then be carried forward into the German translation.

These observations suggest that translation quality control should distinguish errors introduced during recognition from errors introduced during translation. The present procedure compared German output with the system's English ASR text; it was therefore useful for examining translation fidelity but could not independently verify whether the original speech had been recognised correctly.

The quality-control cases were deliberately selected for further examination and were not a representative sample of all translated passages. Moreover, the automated judgements were not calibrated against independent expert annotations, and manual inspection was limited by the author's German proficiency. The resulting counts must therefore not be interpreted as translation error rates or as measures of evaluator accuracy. Their principal contribution was diagnostic: they identified concrete examples of behaviour that the aggregate quality scores did not explain.

5.3 Exploratory Extensions

Two supplementary investigations considered possible alternatives to the main Whisper–Marian cascade. Neither was included in the main controlled comparison.

5.3.1 Whistle as an Alternative ASR Model

The supplementary Whistle experiment examined whether a smaller speech-recognition model could reduce the processing requirements of the cascade. Whistle achieved translation-quality scores broadly comparable to those of `tiny.en`, while using less sampled process memory on the Pi.

However, faster speech recognition did not necessarily translate into a faster complete system. On the Pi, the Whistle configurations achieved total processing RTFs of approximately 0.40–0.42. Most of this processing time was spent in machine translation rather than ASR. The total RTF was therefore higher than for the corresponding `tiny.en` configurations.

This result illustrates the importance of measuring the complete pipeline when evaluating an edge application. Optimising one component does not guarantee an improvement in end-to-end performance when another component becomes the dominant cost.

The Whistle results should nevertheless be treated as a supplementary system comparison rather than a controlled ASR architecture comparison. The models used different inference implementations and optimisation settings, so the observed differences cannot be attributed solely to model size or architecture.

5.3.2 LocalAgreement Prototype

A preliminary extension based on LocalAgreement-2 was also implemented and smoke-tested after the main experiment. LocalAgreement stabilises streaming ASR output by retaining text that agrees between successive recognition hypotheses [5]. Our adaptation translated each newly confirmed English span independently using the existing Marian model.

The smoke test used a 23.6-second recording with `tiny.en`. On the laptop, the fixed 4-second baseline achieved a processing RTF of 0.110 and a chrF2 score of 78.03. With LocalAgreement and minimum update intervals of 2 and 1 seconds, processing RTF increased to 0.298 and 0.683, respectively, while chrF2 decreased to 74.99 and 68.60. English WER remained at 4.92% in all three cases.

The 1-second update configuration produced its first German output approximately 0.82 seconds earlier than the fixed-chunk baseline. This illustrated a potential responsiveness benefit, but it came with substantially greater processing demand and lower translation quality on this recording.

A LocalAgreement run with 2-second updates also completed on the Raspberry Pi, with an RTF of 0.880 and a final output tail of 3.38 seconds. No matched fixed-chunk Pi baseline or corpus-level evaluation was performed for this prototype.

An important implementation question also remained: English spans confirmed by the agreement procedure do not necessarily constitute suitable translation units. Translating each span independently can fragment sentence context and produce awkward German output. A more complete implementation would need to consider how confirmed words are grouped for translation, as well as how latency should be evaluated when update timing depends on computation.

The prototype demonstrated that the approach could run on the Pi, but exposed additional engineering and evaluation requirements without showing an immediate benefit for this cascade. Full integration was therefore left outside the scope of the project. Given the short recording and limited settings, these findings do not establish that fixed chunks generally outperform LocalAgreement.

5.4 Limitations and Future Work

Several limitations affect the interpretation of the results. First, the evaluation covered six recorded talks, totalling approximately 33.6 minutes of unique audio. Although every main configuration was run twice on both devices, the repeated passes used the same recordings. They therefore provide some information about run-to-run consistency but do not increase the diversity of the test material.

Second, the experiment considered only two ASR model sizes and two fixed chunk durations. It did not investigate intermediate chunk sizes, overlapping windows, retained linguistic context, or adaptive segmentation. The findings describe the particular configurations tested, rather than the full range of possible streaming translation approaches.

Third, the resource measurements were limited. Process memory and Pi temperature were sampled periodically, so short-lived peaks could have been missed. The study did not measure electrical power consumption

or battery life, both of which would be relevant to a deployed edge system. The laptop and Pi also differed in architecture, operating system and available processing capacity. Their comparison should consequently be understood as a comparison between two practical deployment environments rather than an isolated test of hardware architecture.

An additional limitation is that the exploratory translation quality-control procedure used the recognised English text as its source. Errors introduced by ASR could therefore be translated faithfully without being identified as translation errors. A more complete evaluation would compare the system's final German output against the original speech, supported by bilingual human assessment.

Future work could examine more context-aware streaming policies, including approaches that combine stable recognition output with larger translation units. The queue events also suggest investigating safeguards against unusually long or repetitive model outputs, which can temporarily delay an otherwise sufficiently fast pipeline.

A larger evaluation with varied speakers and recording conditions would help establish how reliably the system performs beyond the selected MCIF talks. Finally, bilingual human assessment would provide a stronger basis for interpreting translation adequacy and validating the automated quality-control procedure.

Overall, the project demonstrates that a Whisper–Marian speech translation cascade can operate at a sufficient average processing rate on a Raspberry Pi 5. Its practical usefulness, however, depends on more than throughput alone: chunk duration, recognition quality, computation time and occasional processing stalls all contribute to the translation experience.

6

Conclusion

This project investigated the feasibility of running a cascaded streaming speech translation system on a Raspberry Pi 5, with particular attention to the relationship between translation quality, latency and processing requirements.

The first research question concerned how computation time affects the observed quality–latency trade-off. The results showed that computation-aware latency was substantially higher than computation-unaware latency on the Pi. Across the main configurations, computation-unaware LongYAAL ranged from 2.12 to 4.28 seconds, compared with 3.53 to 7.12 seconds when computation was included. This demonstrates the importance of accounting for processing delays when evaluating streaming translation on resource-constrained hardware.

The second research question concerned whether the tested configurations could keep up with incoming speech. All four main Pi configurations achieved an aggregate processing RTF below 1, ranging from 0.228 to 0.516, and completed the test recordings without sustained queue growth. Nevertheless, temporary processing stalls occurred, showing that sufficient average throughput does not guarantee consistently timely output. The laptop processed the same recordings considerably faster.

Longer audio chunks generally improved translation quality but increased latency, while the larger ASR model offered further quality improvements at additional computational cost. The supplementary experiments also illustrated that optimising individual components or adopting a more adaptive streaming policy does not automatically improve end-to-end performance.

Overall, the experiments establish that the tested cascade can process recorded speech at a sufficient average rate on a Raspberry Pi 5. However, they do not establish that the system provides sufficiently responsive or reliable translation for interactive live use.

Bibliography

[1] Luisa Bentivogli et al. “Cascade versus Direct Speech Translation: Do the Differences Still Make a Difference?” In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics, 2021. URL: https://aclanthology.org/2021.acl-long.224/.

[2] Cactus Compute. *Cactus Whistle*. Model card. Released 2 October 2026. 2026. URL: https://huggingface.co/Cactus-Compute/whistle (visited on 10/24/2026).

[3] IWSLT. *Simultaneous ST Track, IWSLT 2026*. Accessed 4 October 2026. 2026. URL: https://iwslt.org/2026/simultaneous.

[4] Guillaume Klein et al. “Efficient and High-Quality Neural Machine Translation with OpenNMT”. In: *Proceedings of the Fourth Workshop on Neural Generation and Translation*. Online: Association for Computational Linguistics, 2020, pp. 211–217. DOI: 10.18653/v1/2020.ngt-1.25. URL: https://aclanthology.org/2020.ngt-1.25/.

[5] Dominik Macháček, Raj Dabre, and Ondřej Bojar. “Turning Whisper into Real-Time Transcription System”. In: *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics: System Demonstrations*. Bali, Indonesia: Association for Computational Linguistics, 2023. URL: https://aclanthology.org/2023.ijcnlp-demo.3/.

[6] Sara Papi et al. “How “Real” is Your Real-Time Simultaneous Speech-to-Text Translation System?” In: *Transactions of the Association for Computational Linguistics* 13 (2025). URL: https://aclanthology.org/2025.tacl-1.14/.

[7] Sara Papi et al. *MCIF: Multimodal Crosslingual Instruction-Following Benchmark from Scientific Talks*. 2025. arXiv: 2507.19634 [cs.CL]. URL: https://arxiv.org/abs/2507.19634.

[8] Peter Polák. *OmniSTEval: Evaluation Toolkit for Simultaneous Speech Translation*. Software. Accessed 4 October 2026. 2025. URL: https://github.com/pe-trik/OmniSTEval.

[9] Peter Polák et al. *Better Late Than Never: Meta-Evaluation of Latency Metrics for Simultaneous Speech-to-Text Translation*. v1 title: "Better Late Than Never: Evaluation of Latency Metrics for Simultaneous Speech-to-Text Translation". 2025. arXiv: 2509.17349 [cs.CL]. URL: https://arxiv.org/abs/2509.17349.

[10] Maja Popović. "chrF: Character n-gram F-score for Automatic MT Evaluation". In: *Proceedings of the Tenth Workshop on Statistical Machine Translation*. Lisbon, Portugal: Association for Computational Linguistics, 2015, pp. 392–395. DOI: 10.18653/v1/W15-3049.

[11] Alec Radford et al. "Robust Speech Recognition via Large-Scale Weak Supervision". In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 28492–28518.

[12] Ricardo Rei et al. "COMET-22: Unbabel-IST 2022 Submission for the Metrics Shared Task". In: *Proceedings of the Seventh Conference on Machine Translation (WMT)*. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, 2022. URL: https://aclanthology.org/2022.wmt-1.52/.

[13] SYSTRAN. *faster-whisper: Faster Whisper Transcription with CTranslate2*. Software. Accessed 4 October 2026. 2023. URL: https://github.com/SYSTRAN/faster-whisper.

[14] Jörg Tiedemann and Santhosh Thottingal. "OPUS-MT – Building Open Translation Services for the World". In: *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation*. Lisboa, Portugal: European Association for Machine Translation, 2020, pp. 479–480. URL: https://aclanthology.org/2020.eamt-1.61/.

[15] TypeSafe. *System One: Jev*. Technical documentation. 2026. URL: https://docs.typesafe.ai/concepts/system-one.